NVIDIA Unreal Engine Streamline Plugin

Table of Contents

Overview

DLSS 4 and the NVIDIA Unreal Engine Streamline plugins

The NVIDIA *Streamline* plugin is part of a wider suite of related NVIDIA performance and image quality improving technologies and

corresponding NVIDIA Unreal Engine plugins:

- NVIDIA DLSS *Frame Generation (DLSS-FG)* boosts frame rates by using AI to render additional frames. *DLSS-FG* requires a GeForce RTX 40 series graphics card.
- NVIDIA DLSS *Multi Frame Generation (DLSS-MFG)* uses AI to boost frame rates by generating up to 3 additional high-quality frames, all while optimizing responsiveness with NVIDIA Reflex. DLSS Multi Frame Generation is available for GeForce RTX 50 Series GPUs.
- NVIDIA DLSS *Super Resolution (DLSS-SR)* boosts frame rates by rendering fewer pixels and using AI to output high resolution frames. *DLSS-SR* requires an NVIDIA RTX graphics card.
- NVIDIA DLSS *Ray Reconstruction (DLSS-RR)* uses AI to enhance image quality and generate additional pixels for intensive ray-traced scenes. Ray Reconstruction replaces hand-tuned denoisers with an NVIDIA supercomputer-trained AI network that generates higher-quality pixels in between sampled rays. This feature is available for all RTX GPUs and requires Super Resolution to be enabled.
- NVIDIA *Deep Learning Anti Aliasing (DLAA)* is used to improve image quality. *DLAA* requires an NVIDIA RTX graphics card.
- NVIDIA *Image Scaling (NIS)* provides best-in class upscaling and sharpening for non-RTX GPUs, both NVIDIA or 3rd party. Please refer to the NVIDIA *Image Scaling* Unreal Engine plugin for further details.
- NVIDIA *RTX Dynamic Vibrance* enhances visual clarity by dynamically tuning color saturation and contrast.
- NVIDIA *Reflex Low Latency* technology, synchronizes the CPU frame submission with GPU processing for just in time rendering.

NVIDIA *DLSS 4* combines DLSS Multi Frame Generation, DLSS Frame Generation, DLSS Super Resolution, DLSS Ray Reconstruction, NVIDIA DLAA, NVIDIA Reflex Low Latency and delivers boosted frame rates, great responsiveness, and better than native IQ.

What's documented in this file

The NVIDIA Unreal Engine Streamline plugins provide:

- DLSS Frame Generation (also called DLSS-G or DLSS-FG)
- DLSS Multi Frame Generation (also called DLSS-MFG)
- NVIDIA Reflex Low latency
- NVIDIA RTX Dynamic Vibrance (also called DeepDVC)

What's available and documented elsewhere

The NVIDIA Unreal Engine DLSS-SR plugin provides:

- DLSS Super Resolution (DLSS-SR)
- Deep Learning Anti-Aliasing (DLAA)
- DLSS Ray Reconstruction (DLSS-RR)
- Movie Render Queue support for DLSS-SR,DLSS-RR, DLAA

The NVIDIA Unreal Engine NIS plugin provides:

- NVIDIA Image Scaling

Quickstart DLSS4 Frame Generation

Packaged engines from the Epic Games Launcher (UE 5.2 and later)

Note that the UE Streamline plugin does not support engine versions before 5.2.

1. Copy the entire Streamline plugins folder and subfolders somewhere under the engine's `Plugins\Marketplace` folder or under your source project's Plugins folder
   - For a packaged engine release from Epic, copy the plugin somewhere under the engine's `Engine\Plugins\Marketplace` folder
   - If you have a source project (not a blueprint-only project) you may also copy the plugin under your project's Plugins folder instead of the engine. Only one copy of the plugin is allowed so don't copy it to both locations
2. Enable the NVIDIA DLSS Frame Generation plugin in the Editor (Edit -> Plugins)
3. Restart the editor
4. Check the log for `NVIDIA Streamline supported 1`

Plugins and Platforms

Windows System requirements for DLSS Frame Generation

- Minimum Windows OS version of Win10 20H1 (version 2004, build 19041 or higher), 64-bit
- Display Hardware-accelerated GPU Scheduling (HWS) must be enabled via Settings : System : Display : Graphics : Change default graphics settings. https://devblogs.microsoft.com/directx/hardware-accelerated-gpu-scheduling/
- NVIDIA Ada architecture GPU (GeForce RTX 40 series, NVIDIA RTX 6000 series)
- NVIDIA GeForce Driver
   - Recommended: version 536.99 or higher

- UE project using DirectX 12 (Default RHI in project settings)

Plugins and other Platforms

The Blueprint modules are supported for all platforms and are stubbed out and return `NotSupported` for the various`QuerySupport` methods. This allows cross platform UI and settings BP and code to compile on all platforms.

The actually platform specific modules (e.g. `StreamlineD3D12RHI`) have the `PlatformAllowList` set to the actually supported platforms, such as `Win64`.

XBOX and UE WinGDK Platform Support

For games that will be released on Xbox, the collection of Streamline UE Plugins is accompanied by a UE platform extension that provides DLSS plugin support for the UE WinGDK platform.

This extension comes in the form of a `StreamlineCore_WinGDK.uplugin` plugin extension and adds support to the StreamlineCore Plugin version 4.0.0 or later.

Note: The per feature Streamline Blueprints plugins (such as StreamlineDLSSG or StreamlineDeepDVC) do **not** need a WinGDK plugin extension since they are platform independent on purpose to allow BP and game code to build and on all platforms.

The WinGDK Platform support files can be found in the `Platforms` folder adjacent to the Plugins folder.

Note that you **must** have the following pre-requisites

- Be a member of the ID@Xbox program and have console access.
- Apply and get approval for Unreal Engine Console Access.

Using the UE WinGDK Extension Plugin

If installing as an engine plugin:

- Copy the files anywhere under the `Engine\Platforms\WinGDK\Plugins` folder

If installing as a project plugin:

- Copy the files anywhere under the project's `Platforms\WinGDK\Plugins` folder

Plugins and Custom Engines

The published UE plugins `EngineVersion` in the respective .uplugin files matching the engine they have been packaged for.

Using a plugin with a mismatched engine (e.g. using the UE 5.6 plugin package with UE 5.5), or with a custom engine that might have non-standard `EngineVersion` fields is not supported.

It will likely succeed to build, but then fail at cook time due to missing plugin Blueprint references.

The solution is to remove the `EngineVersion` field in the plugins when using with a custom engine.

Troubleshooting

Quick tips

1. How to find out which version of DirectX you are using: Open the Output Log in Unreal Editor. Search "RHI" to confirm which version of DirectX you are using. You will see something like "LogD3D12" if it is DX12, for example.
2. How to confirm that Frame Generation initialized correctly: Use the Output Log and search for Streamline, if the initialization succeeded or failed you should see that confirmed here.
3. How to find out which version of Windows you have: Click the Start or Windows button (usually in the lower-left corner of your computer screen). Click Settings. Click System. Click About (usually in the lower left of the screen). The resulting screen shows the edition of Windows.

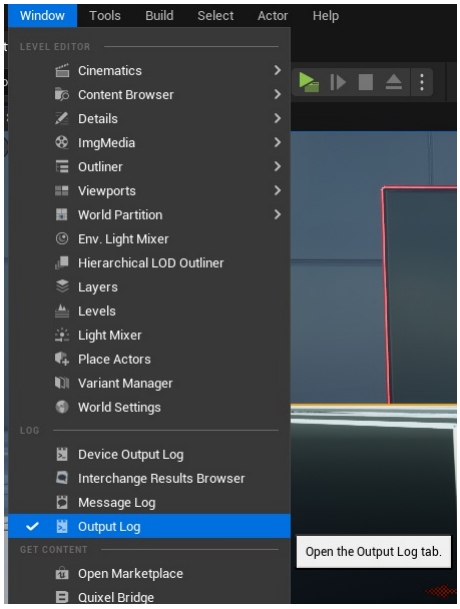
Diagnosing plugin issues in the Editor

The UE DLSS Frame Generation plugin modules write various information into following UE log categories prefixed with *LogStreamline*, e.g.:

- LogStreamline
- LogStreamlineAPI
- LogStreamlineBlueprint
- LogStreamlineD3D11RHI
- LogStreamlineD3D12RHI
- LogStreamlineRHI

- LogStreamlineRHIPreInit

Those can be accessed in the Editor under `Window -> Output Log`.



The Message log then can be filtered to show only the Streamline related messages to get more information on why the plugin might not be functioning as expected.

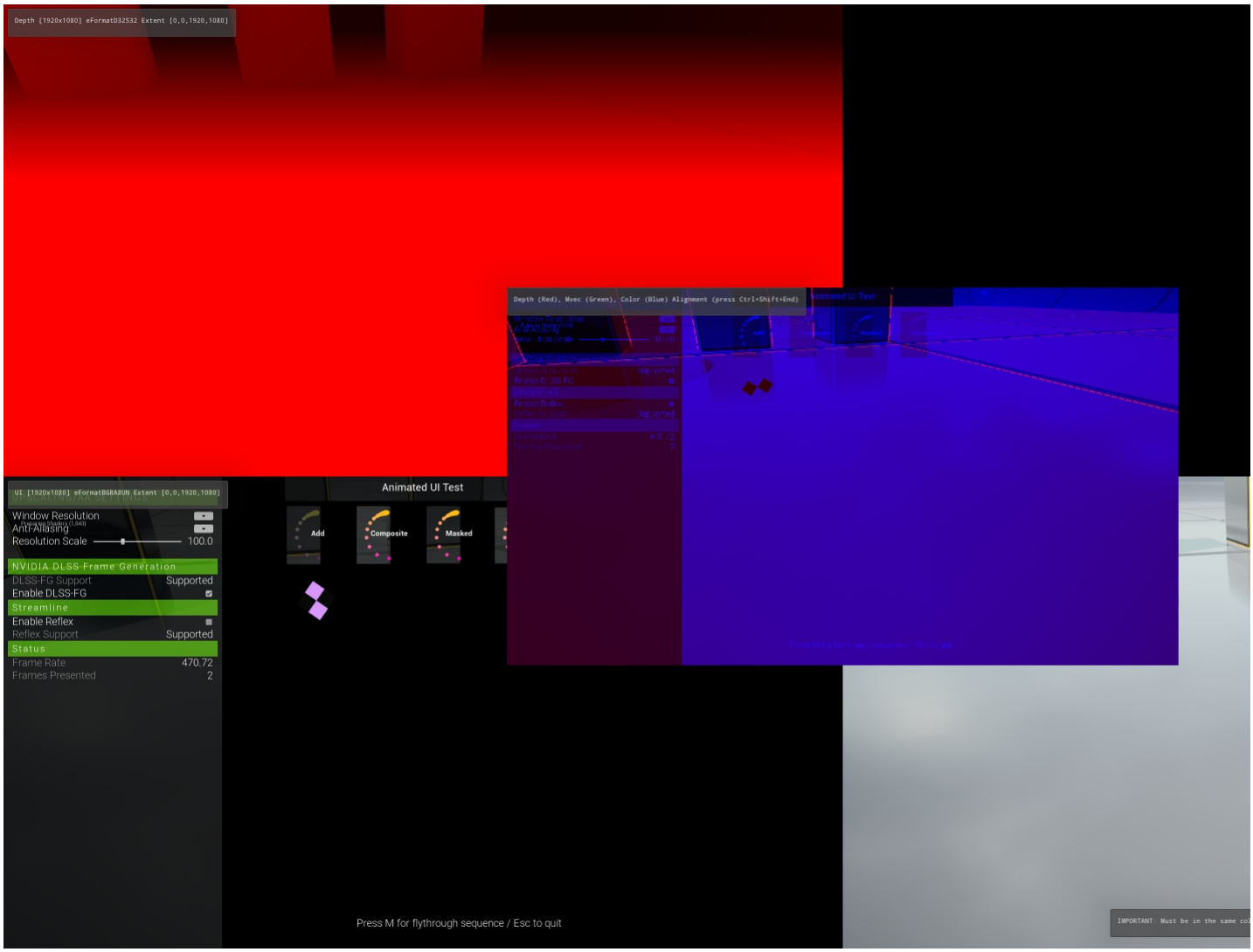
Debug overlay

In non-shipping UE builds, the DLSS Frame Generation plugin can show a debug overlay that displays runtime information. The overlay can be enabled/disabled in non Shipping builds with the `Plugins -> NVIDIA DLSS Frame Generation -> Load Debug Overlay` option in Project Settings.

This setting can also be overridden with the `-sldebugoverlay` and `-slnodebugoverlay` on the command line. This implicitly selects Streamline Development binaries.

The overlay is only supported on systems where the DLSS-FG feature is supported.

The debug overlay provides a way to visualize the various textures passed into Streamline for DLSS-FG. With DLSS-FG enabled `Ctrl+Shift+Insert` enables the following image view.



Incompatibilities with the Mobile Forward Renderer

This plugin is only compatible with the deferred renderer. The mobile renderer is not supported since it doesn't call the relevant `FSceneViewExtension` methods needed.

Incompatibilities with Non-Cooker Commandlets

This plugin disables the runtime components when run in commandlet (e.g. WorldPartitionBuilderCommandlet), regardless of whether -AllowRendering is specified or not.

Graphics Debug and Validation Layers

The UE plugins connect the various UE RHIs, the underlying graphics APIs and NVIDIA SDKs. As such they are exposed to changes across RHIs across UE versions.

The underlying hard and software stack is pretty indifferent to the resource states of modern graphics APIs, but consistent resource state between the engine/RHI, the graphics API and the NVIDIA SDK's is needed for compatibility with graphics API debug layers.

This section provides an overview over those parts of the UE plugins, please refer to the source and comments for further details.

UE 5.6 and newer

UE 5.6:

- Moved the majority of the resource state management from the D3D12RHI to the RDG, removing the D3D12RHI resource state tracker.
- Adds `ID3D12DynamicRHI::RHIUpdateResourceResidency` that allows plugins to make input and output resources resident.
  - This avoids GPU memory access faults when the system is under memory pressure.
- Adds `ID3D12DynamicRHI::RHIFlushResourceBarriers` that allows plugins to flush pending resource barriers of the D3D12RHI to be submitted to the command list.
  - This allows D3D12 resource states match between the RHI, the DX12 API and the NVIDIA SDK, thus avoiding D3D debug layer error.

UE 5.5 and older

UE 5.5 and older:

- Has resource state management split between the RDG and the D3D12RHI resource state tracker.
- Do not support `ID3D12DynamicRHI::RHIUpdateResourceResidency`.
  - In 5.5 particular this can lead to lost devices / GPU memory fault when the system is under memory pressure because the plugin resources are not made D3D12 resident before calling into the NVIDIA SDK.
- Do not support `ID3D12DynamicRHI::RHIFlushResourceBarriers`.
  - This means that it's impossible to pass the actual D3D12 API resource state correctly into the NVIDIA SDKs. This then causes d3d debug validation layer messages.

However the effects (residency management and pending barrier flushing) of those UE 5.6 `ID3D12DynamicRHI` methods can be achieved by ~~abuses~~ side-effects of existing UE 5.5 `ID3D12DynamicRHI` methods that call the D3D12RHI side internal methods that are exposed directly via the 5.6 methods:

- `ID3D12DynamicRHI::RHIUpdateResourceResidency` can be achieved by calling UE 5.5 `ID3D12DynamicRHI::RHITransitionResource` on the plugin input and output resources
  - Calling those methods also puts the resources into a consistent D3D12 resource state at the RHI level that then can be passed into the NVIDIA SDK's. However those resource transitions are pending at the D3D12RHI and need to be flushed, as discussed below.
- `ID3D12DynamicRHI::RHIFlushResourceBarriers` can be achieved by calling UE 5.5 `RHICopyTexture` on unrelated textures.
  - This creates additional RDG passes and resources that have resource labels and draw event names like `UE5.5AndOlderDebugLayerCompatibilityHelperSource` or `UE5_5AndOlderBackdoorViaUnrelatedCopy` that should make them easy to spot in debugging tools such as Nsight Graphics
  - Since this is only needed for D3D debug layer compatibility and not functionality, those extra passes are only executed when the d3d debug layer is enabled by the engine, either via `-d3ddebug` or the respective config file section.
  - **Note:** `-d3ddebug` is also supported in `UE_BUILD_SHIPPING`, and so are the additional debug layer compatibility passes.

Experimental Backports from 5.6 `ID3D12DynamicRHI` methods to 5.5 and older

If those trade-offs from above are insufficient for a particular use case, the `ID3D12DynamicRHI::RHIFlushResourceBarriers` and `ID3D12DynamicRHI::RHIUpdateResourceResidency` are isolated enough that they could be backported to 5.5 and older if needed.

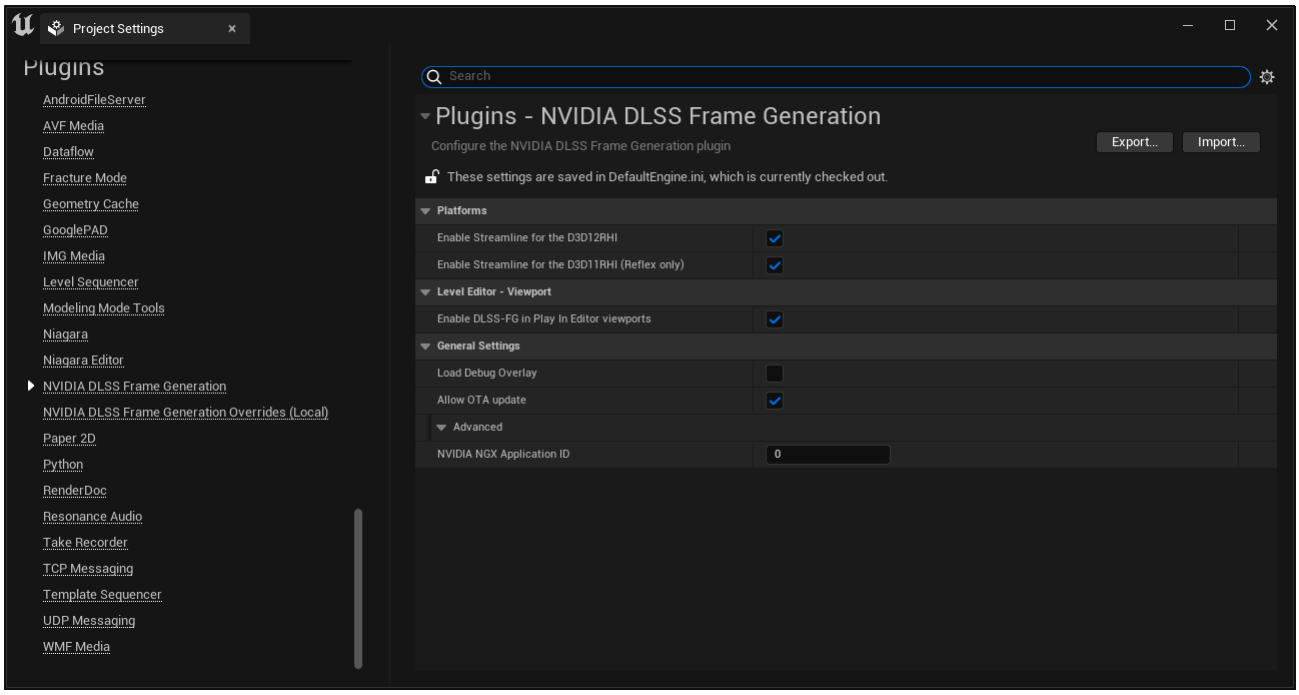
1. Cherry-pick `2be94f9e1642ca026c07cbcc980ac922afc17b00 Add RHIUpdateResourceResidency API to ID3D12DynamicRHI`
2. Cherry-pick `642d7f4108155528627f27eec161f06936d97659 Allow DX12 Resource barrier flushing trough ID3D12DynamicRHI`
3. Add to the bottom of `RHIDefinitions.h` (and not `ID3D12DynamicRHI.h`): `#define ENGINE_PROVIDES_UE_5_6_ID3D12DYNAMICRHI_METHODS 1`
4. The UE plugin guards relevant code paths by `ENGINE_PROVIDES_UE_5_6_ID3D12DYNAMICRHI_METHODS`

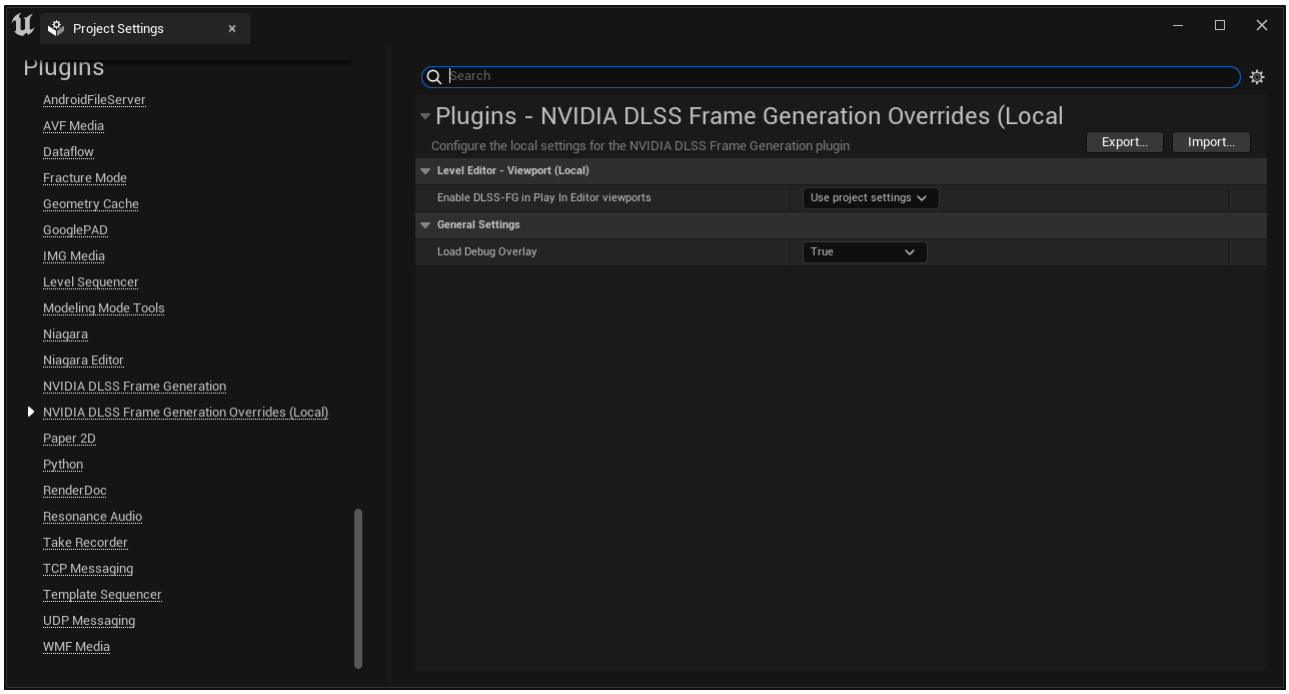
Note: This is path is provided as-is and might require adjustments

Plugin configuration

Some project settings are split across two config files, with the possibility of a local override.

- `Project Settings -> Plugins -> NVIDIA DLSS Frame Generation`
  - stored in DefaultEngine.ini
  - typically resides in source control.
  - settings here are shared between users
- `Project Settings -> Plugins -> NVIDIA DLSS Frame Generation Overrides (Local)`
  - stored UserEngine.ini
  - not recommended to be checked into source control.
  - allow a user to override project wide settings if desired. Defaults to "Use project settings"





Command Line Options And Console Variables and Commands

Note: UE DLSS Frame Generation plugin modules are loaded very early during engine startup, before console variables are available. As such, various debugging settings are exposed via command line options instead of console variables.

Selecting Streamline binary flavor

By default Streamline uses signed production binaries (e.g. `sl.interposer.dll`, `sl.common.dll`) from the `Streamline\Binaries\ThirdParty\Win64\` path. They have no on-screen watermarks and are intended to be used for applications shipped to end users. They are always packaged by the UE Streamline plugin.

For non-shipping UE builds, alternate binaries can be selected via `-slbinaries={development,debug}` command line argument, corresponding to the `Streamline\Binaries\ThirdParty\Win64\Development` and `Streamline\Binaries\ThirdParty\Win64\Debug` paths respectively. They have on-screen watermarks and are intended to be used during application development and are a requirement to use the Streamline debugoverlay. They are only packed by the UE Streamline plugin for non-shipping UE builds.

Alternatively, `-slbinaries=<path>` can be used to load Streamline binaries from an arbitrary directory (e.g. `-slbinaries=D:/my/custom/sl/binaries`). If the argument is an existing directory path, binaries are loaded directly from that location instead of the plugin's bundled paths.

Logging

By default Streamline uses `sl:eLogLevelDefault`. This can be changed with the `-sllloglevel={0,1,2}`command line argument, corresponding to `sl::eLogLevelOff`, `sl::eLogLevelDefault`, `sl::eLogLevelVerbose` respectively

By default the Streamline console window is turned off. This can be changed wit the `-sllogconsole={0,1}` command line argument.

Streamline Common Settings

The UE Streamline plugin uses various engine side hooks, which can be configured by the following cvars.

Console Variables (low level)

- r.Streamline.ViewIndexToTag
  - Which view of a view family to tag
  - -1: -1: all views
  - 0: first view (default)
  - 1..n: nth view, typically up to 3 when having 4 player split screen view families
- r.Streamline.ViewIdOverride
  - Replace the view id passed into Streamline based on)
  - -1: Automatic, based on the state of r.Streamline.ViewIndexToTag (default)
  - 0: use ViewState.UniqueID
  - 1: override to 0
- r.Streamline.TagSceneColorWithoutHUD
  - Pass scene color without HUD into DLSS Frame Generation (default = true)
- r.Streamline.Editor.TagSceneColorWithoutHUD
  - Pass scene color without HUD into DLSS Frame Generation in the editor (default = true)
- r.Streamline.ClearSceneColorAlpha
  - Clear alpha of scenecolor at the end of the Streamline view extension to allow subsequent UI drawcalls be represented correctly in the alpha channel (default=true)
- r.Streamline.Editor.TagUIColorAlpha
  - Experimental: Pass UI color and alpha into Streamline in Editor PIE windows (default = false)
- r.Streamline.TagBackbuffer
  - Pass backbuffer extent into Streamline (default = true)
- r.Streamline.MaxNumSwapchainProxies
  - Determines how many Streamline swapchain proxies can be created. This impacts compatibility with some Streamline features that have restrictions on that\n")
  - -1: automatic, depending on enabled Streamline features (default)
  - 0: no swap chain proxy. Likely means features needing one won't work
  - 1..n: only create a Streamline swapchain proxy for that many swapchains/windows,
- r.Streamline.FilterRedundantSetOptionsCalls
  - Determines whether the UE plugin filters redundant calls into
  - 0: call every streamline sl{Feature}SetOptions function, regardless of whether UE plugin side changed or not. Helpful for debugging. Can also be override with -sl{no}filter command line option
  - 1: only call sl{Feature}SetOptions when the UE plugin side changed(default)

Controlling which Streamline feature is loaded at startup

There are a few cvars that control whether a specific Streamline feature is loaded at startup time. Selectively turning those off in a config file and restarting allows to implicitly control which UE engine hooks the Streamline plugin registers.

E.g. turning DLSS-FG off that way will prevent the Streamline plugin register a `IDXGISwapchainProvider` which might be used by 3rd party UE plugins as well.

- Determines whether feature %s is loaded. This can be useful to resolve conflicts where multiple SL features are incompatible with each other.
  - r.Streamline.Load.Reflex

- r.Streamline.Load.DLSSG
- r.Streamline.Load.DeepDVC

Reflex (Streamline)

The UE DLSS Frame Generation plugin provides an implementation of NVIDIA Reflex, which is semi-compatible with the existing UE Reflex plugin that ships with unmodified versions of UE.

It is recommended to disable the existing UE Reflex plugin and use the Reflex implementation provided in the UE DLSS Frame Generation plugin. But existing projects using blueprint functions from the UE Reflex plugin should continue to work after adding the UE DLSS Frame Generation plugin, because the Reflex plugin's blueprints should call into the DLSS Frame Generation plugins's modular features by default.

Frame rate limiting

Due to how Unreal Engine handles frame rate limiting, there may be issues where the frame rate gets stuck at the minimum frame rate and doesn't recover. The console variable `t.Streamline.Reflex.HandleMaxTickRate` can be set to `False` to let the engine limit the max tick rate instead of Streamline Reflex, which may help these situations. The engine is unaware of DLSS frame generation, so the actual graphics frame rate may be effectively double what the engine's max FPS is set to when this cvar is `False`.

Reflex stats

When Reflex or DLSS Frame Generation are enabled, Reflex may handle delaying the game thread to enforce any frame rate limits, such as those requested through `t.MaxFPS` or frame rate smoothing. The "Game thread wait time (Reflex)" stat of `stat threading` shows the time Reflex spent delaying the game thread.

Reflex Blueprint library

The `Streamline|Reflex/UStreamlineLibraryReflex` blueprint library is the recommended way to query whether Reflex is supported and also provides functions to configure Reflex

- `IsReflexSupported`, `QueryReflexSupport`,
- `IsReflexModeSupported`, `GetSupportedReflexModes`
- `SetReflexMode`, `GetReflexMode`, `GetDefaultReflexMode`
- `GetGameToRenderLatencyInMs`, `GetGameLatencyInMs`,`GetRenderLatencyInMs`

Console Variables (low level)

It can be configured with the following console variables

- t.Streamline.Reflex.Enable
  - Enable Streamline Reflex extension. (default = 0)
  - 0: Disabled
  - 1: Enabled
- t.Streamline.Reflex.Auto
  - Enable Streamline Reflex extension when other SL features need it. (default = 1)
  - 0: Disabled
  - 1: Enabled
- t.Streamline.Reflex.EnableInEditor
  - Enable Streamline Reflex in the editor. (default = 1)
  - 0: Disabled
  - 1: Enabled
- t.Streamline.Reflex.Mode
  - Streamline Reflex mode (default = 1)
  - 0: off
  - 1: low latency
  - 2: low latency with boost
- t.Streamline.Reflex.HandleMaxTickRate
  - Controls whether Streamline Reflex handles frame rate limiting instead of the engine (default = true)
  - false: Engine handles frame rate limiting
  - true: Streamline Reflex handles frame rate limiting

Interactions between the UE `Reflex` plugin and Reflex provided by the `DLSS Frame Generation` plugin can be configured with this console variable:

- r.Streamline.UnregisterReflexPlugin
  - The existing NVAPI based UE Reflex plugin is incompatible with the DLSS Frame Generation based implementation. This cvar controls whether the Reflex plugin should be unregistered from the engine or not
  - 0: keep Reflex plugin modular features registered
  - 1: unregister Reflex plugin modular features. The Reflex blueprint library should work with the DLSS Frame Generation plugin modular features (default)

DLSS Frame Generation

The UE DLSS Frame Generation plugin provides an implementation of NVIDIA DLSS Frame Generation (DLSS-FG).

DLSS-FG is supported fully in packaged builds (or running the editor in -game mode)

DLSS-FG is partially supported in the editor (**UE4 not supported**), with the following limitations:

- The main editor window does *not* support DLSS-FG. So "play in selected viewport" is *not* supported.
- PIE (new Window) is supported for a single PIE window.
  - When multiple PIE clients are configured (PlayNumberOfClients > 1), DLSS-FG is disabled entirely for all PIE windows.
  - Use the "Enable DLSS-FG in New Editor Window (PIE) mode" option in the project settings to enable/disable this

Note: DLSS-FG also needs Reflex to be enabled for optimal performance. This is done automatically. See t.Streamline.Reflex.Auto for further details

DLSS Multi Frame Generation

The DLSS Multiframe Generation modes have been added as `3X` and `4X` members to `EStreamlineDLSSGMode`. `2X`corresponds to the `On` mode of the 1.x.x release.

Any existing BP or C++ code that has been using `GetSupportedDLSSGModes` and `UStreamlineLibraryDLSSG::SetDLSSGMode` to implement UI should require no additional effort (other than maybe localization) to have DLSS-MFG be available in a project.

Content Considerations

Correct rendering of UI alpha

DLSS-FG can make use of a UI color and alpha buffer to increase image quality of composited UI. The UE DLSS Frame Generation plugin generates this from the backbuffer by extracting the alpha channel after UI has been rendered (right before present).

UE by default does not clear alpha of the scenecolor, which works fine since most, if not all UMG material blending modes only consider incoming alpha and write all pixels. However the developer console window gets rendered after the UI elements, writing to the alpha channel. This alpha persist across frames and will result in incorrect UI color ands alpha buffer being generated by the UE Streamline plugin and then passed into Streamline/DLSS-FG.

The engine does not provide a built-in way to clear just the alpha of the scenecolor. Thus the UE Streamline plugin adds a renderpass to clear the alpha channel of the scene color at the end of postprocessing, before UI elements get rendered. This is controlled by `r.Streamline.ClearSceneColorAlpha`, which is true by default.

The Streamline plugin also adds threshold to determine a pixel as UI where the pixel has larger alpha than the threshold, `r.Streamline.TagUIColorAlphaThreshold`. By default, the cvar is set to 0.0 so that any pixel which has alpha greater than zero, is extracted to UI color and alpha buffer.

Note: UI color alpha support is only supported in standalone game windows. In Editor PIE popup windows UI color and alpha tagging disabled by default (see `r.Streamline.Editor.TagUIColorAlpha`) since clearing the alpha channel is not trivial: PIE windows render the scene into a separate 'BufferedRT' render target, blit that into the backbuffer and then draw UI on top of that before presenting. This 'BufferedRT' intermediate step prevents `r.Streamline.ClearSceneColorAlpha` from working as intended. As such image quality in the PIE editor windows is not representative, however should be sufficient to develop UI and settings logic using the blueprint library.

This however only works if the application renders UI in the expected way, which means any UI element needs to write alpha into the backbuffer (regardless of whether it's opaque or transparent). In practice this means:

- Don't use `UWidgetBlueprintLibrary::DrawText` to draw UI text since that does *not* write alpha.
- Do use UMG widgets instead since those correctly write alpha into the backbuffer with an appropriate blending mode

Full screen menus

It is recommended to disable DLSS-FG in full screen menus. It's best if this is handled in application logic since the application should know when a full screen menu is active or not. But as an alternative, setting the console variable `r.Streamline.DLSSG.FullScreenMenuDetection` to `true` will enable automatic full screen menu detection, which will attempt to disable DLSS-FG when a full screen menu is active and re-enable DLSS-FG when the full screen menu is no longer active. Automatic detection is disabled by default.

HDR and UI Color and Alpha

In HDR mode the engine blends UI differently and as such the backbuffer alpha channel does *not* contain useful alpha values for DLSS-

FG, they are actually harmfull and reduce image quality.

When rendering with HDR, it is recommended to disable UI color & alpha tagging by setting `r.Streamline.TagUIColorAlpha` to 0.

Possible Reduced IQ with OpenColorUI plugin

The OpenColor UI plugin provides a sophisticated tonemapper that however can produce negative numbers as its output.

DLSS-FG however expects any of it's inputs to not have any negative numbers.

Negative inputs can yield reduced image quality such as chunks/parts of foliage lagging behind the rest of the image.

It is recommended to either disable the OpenColorIO plugin when used with DLSS-FG or add manually shader passes to remove negative values and others such as InF or NaN from the DLSS-FG inputs

Fine-tuning motion vectors for DLSS Frame Generation

- r.Streamline.DilateMotionVectors
  - 0: pass low resolution motion vectors into DLSS Frame Generation (default)
  - 1: pass dilated high resolution motion vectors into DLSS Frame Generatio. This can help with improving image quality of thin details.
- r.Streamline.MotionVectorScale
  - Scale DLSS Frame Generation motion vectors by this constant, in addition to the scale by 1/ the view rect size. (default = 1.0)

Fine-tuning depth for DLSS Frame Generation

- r.Streamline.CustomCameraNearPlane
  - Custom distance to camera near plane. Used for internal DLSS Frame Generation purposes, does not need to match corresponding value used by engine. (default = 0.01)
- r.Streamline.CustomCameraFarPlane
  - Custom distance to camera far plane. Used for internal DLSS Frame Generation purposes, does not need to match corresponding value used by engine. (default = 75000.0)

DLSS-FG auto mode

It is recommended to use Auto mode when enabling DLSS-FG with the "Set DLSS-FG Mode" Blueprint node. In Auto mode, DLSS-FG will be automatically disabled in cases where it would reduce the frame rate. When enabling Auto mode for the first time in an application, you may notice DLSS-FG disabled for about 2 seconds shortly after it is enabled. This is normal behavior during initial auto mode calibration. Auto mode can also be enabled with the console command `r.Streamline.DLSSG.Enable 2`.

Framerate and Presented Frames

The engine is not directly aware of additional frames generated by frame generation, so built-in frame rate and frame time metrics may be missing information. The "DLSSG" stat group provides frame rate stats taking into account the additional frames. Use the console command `stat dlssg` to see on-screen information.

Tips and Best Practices

1. Once Streamline/Frame Generation is active, you can activate it in a PIE or Standalone window within the editor by typing the command `r.streamline.dlssg.enable 1` or by using the Blueprint Script function (search for "streamline" to get a list of these functions) and enabling it on beginplay.

2. Navigate to project settings, and then to your preferences for the "NVIDIA DLSS Frame Generation" plugin. Here, toggle the "Load Debug Overlay" option for a fast and convenient way to confirm that DLSS Frame Generation is working, along with real time statistics.

3. In this same settings window, ensure the "Allow OTA Update" option is enabled, which will automatically update Streamline as well as DLSS's AI algorithms with the latest improvements.

4. Note that the Debug Overlay for Frame Generation will work in the editor and can appear in Development/Debug builds, but won't appear in Shipping builds.

5. In the Unreal Editor, Frame Generation only works from a New Editor Window (PIE) or Standalone mode, it doesn't work from the Selected Viewport or while editing.

6. Vsync support can be queried via GetDLSSGIsVsyncSupportAvailable BP function.

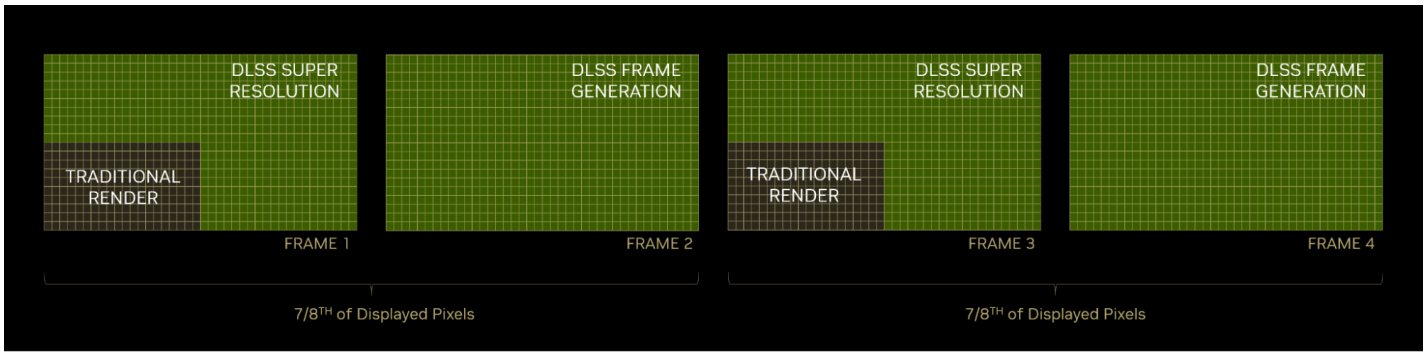
7. The NVIDIA DLSS Frame Generation Unreal Engine plugin contains the latest NVIDIA Reflex technology – a newer version than the Reflex currently built into Unreal Engine. While it's possible to keep the old plugin enabled, and even use the old Reflex

Blueprint Scripts, it's recommended that you disable the old Reflex plugin and use the new version bundled in DLSS Frame Generation instead.

8. We recommend you set up all NVIDIA plugins via Blueprint scripts, as this allows you to conveniently activate plugins from UI menus and set preferences for users. However, if you need access to the DLSS Super Resolution console commands, they can be found under 'ngx', while DLSS Frame Generation commands can be found under 'streamline'. Please read the DLSS_Super_Resolution_UE_Plugin_Guide.pdf included in the DLSS 4 plugin download for more information on using DLSS Super Resolution console commands.

Expectation of performance benefits

DLSS 4 is the performance multiplier. When activated it has the potential to dramatically increase framerate in many situations.



However there are some notes and caveats to that, so be aware of the following:

1. DLSS Super Resolution may not increase framerate in Unreal Engine on its own. That's because UE's default behavior is to use upscaling and manage that upscaled resolution automatically depending on the window size. So when you turn on DLSS Super Resolution you will get the benefits of AI enhanced upscaling, but it will use the same input resolutions as the default.
2. That being said, you may find that DLSS SR can use lower input resolutions to get the same effective quality, and DLSS supports input resolutions all the way down to 33% (Ultra performance) mode.
3. Depending on the scene and input resolution, performance multipliers of anywhere from 1.5x to 3x or more can be expected.
4. Complex scene running at 100% native resolution, 4k, 16fps

5. Same scene running at 33% DLSS resolution, 4k, 60fps

6. DLSS SR may not gain much framerate in CPU bound situations, because the GPU will be starved for work. To get the maximum performance from DLSS SR you should try to alleviate any CPU related bottlenecks that may be preventing it from working at its highest capacity.
7. DLSS Frame Generation can help to alleviate many CPU and GPU bound situations, the performance gain can be anywhere from 1.5x to 2.2x or more.
8. Same scene running at 33% DLSS resolution and Frame Generation, 4k, 100fps

9. Note that both DLSS SR and FG have some initial cost to set up, and this cost will depend on the scene and resolution. Typically these performance costs can be anywhere from 0.5 to 2+ milliseconds, but the goal is that the net performance gain will outweigh the costs.

DLSS-FG Blueprint library

The `Streamline|DLSS-FG/UStreamlineLibraryDLSSG` blueprint library is the recommended way to query whether DLSS-FG and DLSS-MFG is supported and also provides functions to configure DLSS-FG

- `IsDLSSGSupported`, `QueryDLSSGSupport`, `GetDLSSGMinimumDriverVersion`
- `IsDLSSGModeSupported`, `GetSupportedDLSSGModes`
- `SetDLSSGMode`, `GetDLSSGMode`, `GetDefaultDLSSGMode`, `GetDLSSGFrameTiming`

Console Variables (low level)

It can be configured with the following console variables

- r.Streamline.DLSSG.Enable
  - DLSS-FG mode (default = 0)
  - 0: off
  - 1: always on
  - 2: auto mode (on only when it helps frame rate)
  - 3: dynamic, if available otherwise reverts to auto
- r.Streamline.DLSSG.FramesToGenerate
  - Number of frames to generate (default = 1)
  - 1..3:
- r.Streamline.DLSSG.AdjustMotionBlurTimeScale
  - When DLSS-G is active, adjust the motion blur timescale based on the generated frames
  - 0: disabled
  - 1: enabled, not supporting auto mode
  - 2: enabled, supporting auto mode by using last frame's actually presented frames (default)
- r.Streamline.TagUIColorAlpha
  - Pass UI color and alpha into DLSS Frame Generation (default = true)
- r.Streamline.DLSSG.FullScreenMenuDetection
  - Automatically disable DLSS-FG if full screen menus are detected (default = false)
- r.Streamline.DLSSG.DynamicResolutionMode
  - Experimental: Pass in sl::DLSSGFlags::eDynamicResolutionEnabled (default = 0)
  - 0: off
  - 1: on
- r.Streamline.DLSSG.RetainResourcesWhenOff
  - Instruct DLSS-G to not release resources when turned off (default = false)

Known Issues

- With older drivers, possibility of "device removed" error when enabling DLSS-FG if SER (Shader Execution Reordering) is active at the same time. Recommend at least driver 536.99 to avoid this issue

RTX Dynamic Vibrance (DeepDVC)

DeepDVC Blueprint library

The `Streamline|Deep DVC/UStreamlineLibraryDeepDVC` blueprint library is the recommended way to query whether DLSS-FG and DLSS-MFG is supported and also provides functions to configure DLSS-FG

- `IsDeepDVCSupported`, `QueryDeepDVCSupport`, `GetDeepDVCMinimumDriverVersion`
- `IsDeepDVCModeSupported`, `GetSupportedDeepDVCModes`
- `SetDeepDVCMode`, `GetDeepDVCMode`, `GetDefaultDeepDVCMode`
- `SetDeepDVCIntensity`, `GetDeepDVCIntensity`
- `SetDeepDVCSaturationBoost`, `GetDeepDVCSaturationBoost`

Console Variables (low level)

- r.Streamline.DeepDVC.Enable
  - DeepDVC mode (default = 0) -0: off
  - 1: always on
- r.Streamline.DeepDVC.Intensity"),
  - DeepDVC Intensity (default = 0.5, range [0..1])
  - Controls how strong or subtle the filter effect will be on an image. A low intensity will keep the images closer to the original, while a high intensity will make the filter effect more pronounced.
    - Note: '0' disables DeepDVC implicitly
- r.Streamline.DeepDVC.SaturationBoost
  - DeepDVC SaturationBoost(default = 0.5) [0..1]
  - Enhances the colors in them image, making them more vibrant and eye-catching. This setting will only be active if r.Streamline.DeepDVC.Intensity is relatively high. Once active, colors pop up more, making the image look more lively.
  - Note: Applied only when r.Streamline.DeepDVC.Intensity > 0